

The logo of K. J. Somaiya College of Engineering is a shield-shaped emblem. It features a central figure holding a torch, surrounded by various symbols including a gear, a book, and a lamp. The text "K. J. SOMAIYA COLLEGE OF ENGINEERING" is inscribed within the shield.

Experiment No.: 05

Title: Continuous Integration with Jenkins and GitHub

Batch: B-1**Roll No.: 16010422234****Experiment No.: 05**

Aim: Continuous Integration with Jenkins, Git and GitHub for each build.

Resources needed: Jenkins, GitHub Account

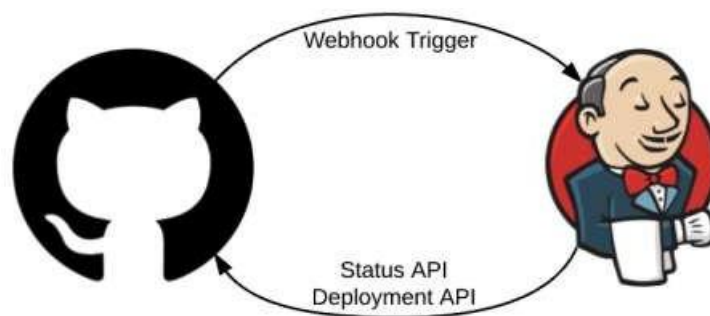
Theory:

Implementing Continuous Integration (CI) can reduce errors in code, shorten development cycles, and help your team ship software faster.

Why does CI matter?

Companies need to release software faster than ever, and CI has become an important part of keeping up. With CI in place, teams can build, test, and update their code within minutes, increasing quality and reducing time to production.

With so much change in the tech field, teams can use many different kinds and combinations of tools. That's why our focus is on making life easier for as many developers as possible, giving teams the freedom to integrate with the best tools out there. Today, GitHub integrates with hundreds of third party tools, including some of the most popular and well-documented integrations around—all available from Marketplace and the Works with GitHub directory.

How does it work with GitHub?

Jenkins scans your entire GitHub organization and creates Pipeline jobs for each repository containing a [Jenkinsfile](#) — a text file that defines the process of building, testing, and deploying your project using Jenkins. Immediately after code is checked in or a new pull request is created, Jenkins will execute the Pipeline job and return the status to GitHub indicating whether it failed or succeeded. This process allows you to run a build and subsequent automated tests at every check in so only the best code gets merged. Catching bugs early and automatically reduces the number of problems introduced into production, so your team can build better, more efficient software.

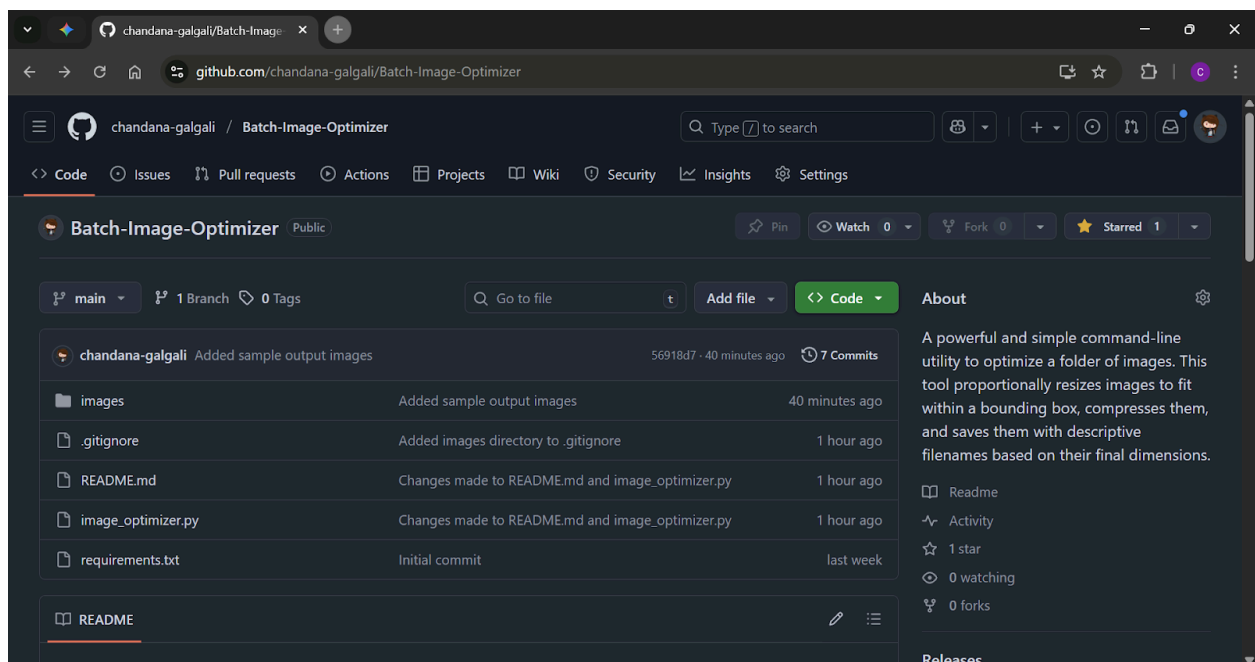
Deployments that occur in Jenkins can also be recorded back into GitHub to audit the entire lifecycle.

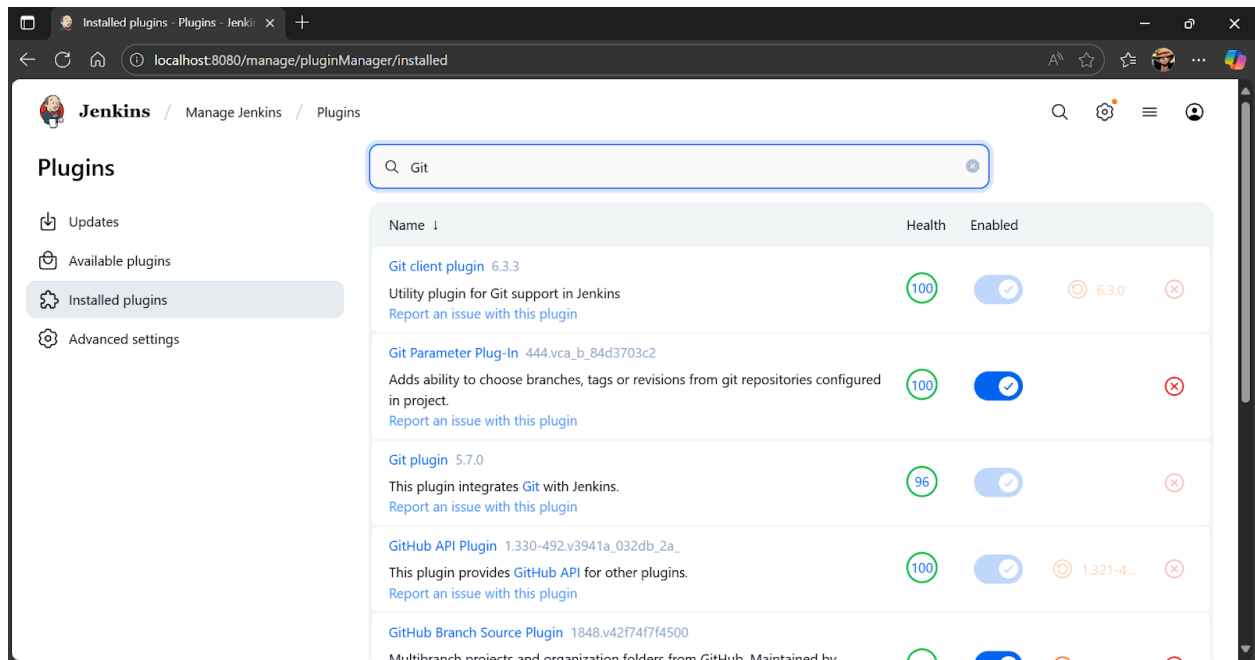
Procedure:

1. Write any program and push it on GitHub
 2. Integrate Jenkins and GitHub
 - a. Install Git plugin on Jenkins
 - b. Create a project on Jenkins and configure it for Git SCM
 3. Run Git Push command to commit the modified code on GitHub
 4. Check the changes on the GitHub and Jenkins for continuous integration for each build.
-

Results: (Document with screenshots)

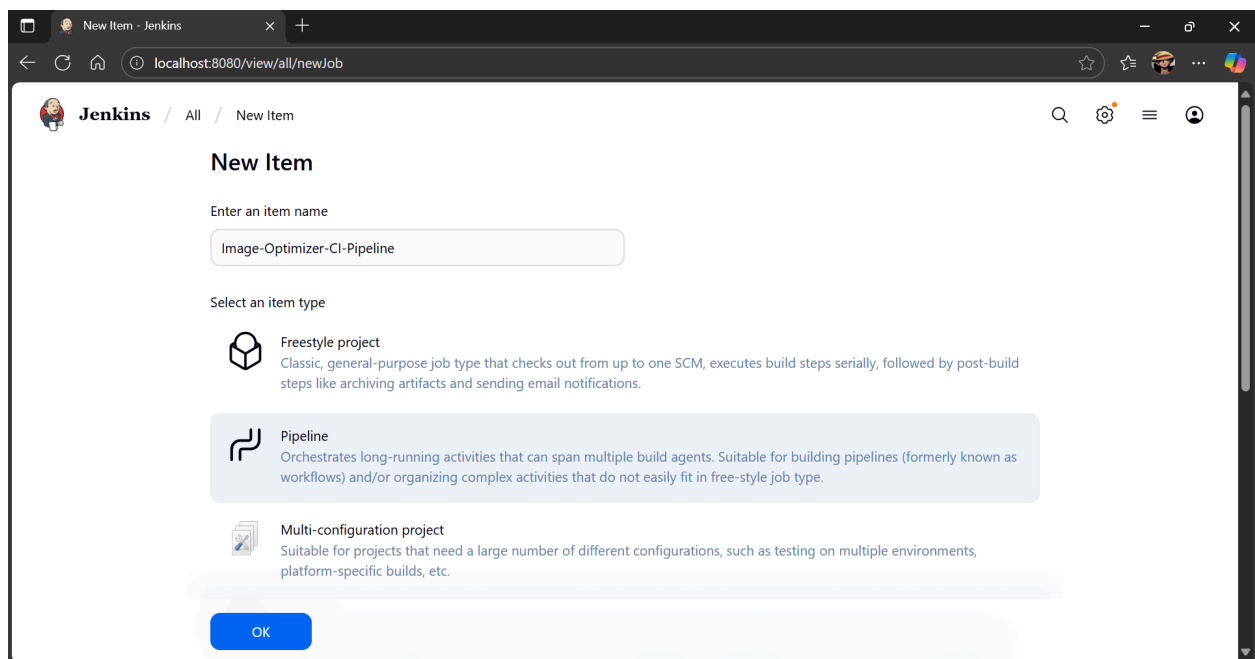
- A. All steps of GitHub integration, configuration and execution with modification of code for each build.





The screenshot shows the Jenkins 'Installed plugins' page. The browser address bar indicates the URL is `localhost:8080/manage/pluginManager/installed`. The Jenkins header shows 'Jenkins / Manage Jenkins / Plugins'. A search bar contains the text 'Git'. On the left sidebar, 'Installed plugins' is selected. The main content area displays a table of installed plugins:

Name	Health	Enabled
Git client plugin 6.3.3 Utility plugin for Git support in Jenkins Report an issue with this plugin	100	☒
Git Parameter Plug-In 444.vca_b_84d3703c2 Adds ability to choose branches, tags or revisions from git repositories configured in project. Report an issue with this plugin	100	☒
Git plugin 5.7.0 This plugin integrates Git with Jenkins. Report an issue with this plugin	96	☒
GitHub API Plugin 1.330-492.v3941a_032db_2a_ This plugin provides GitHub API for other plugins. Report an issue with this plugin	100	☒
GitHub Branch Source Plugin 1848.v42f74f7f4500 Multibranch projects and organization folders from GitHub. Maintained by		

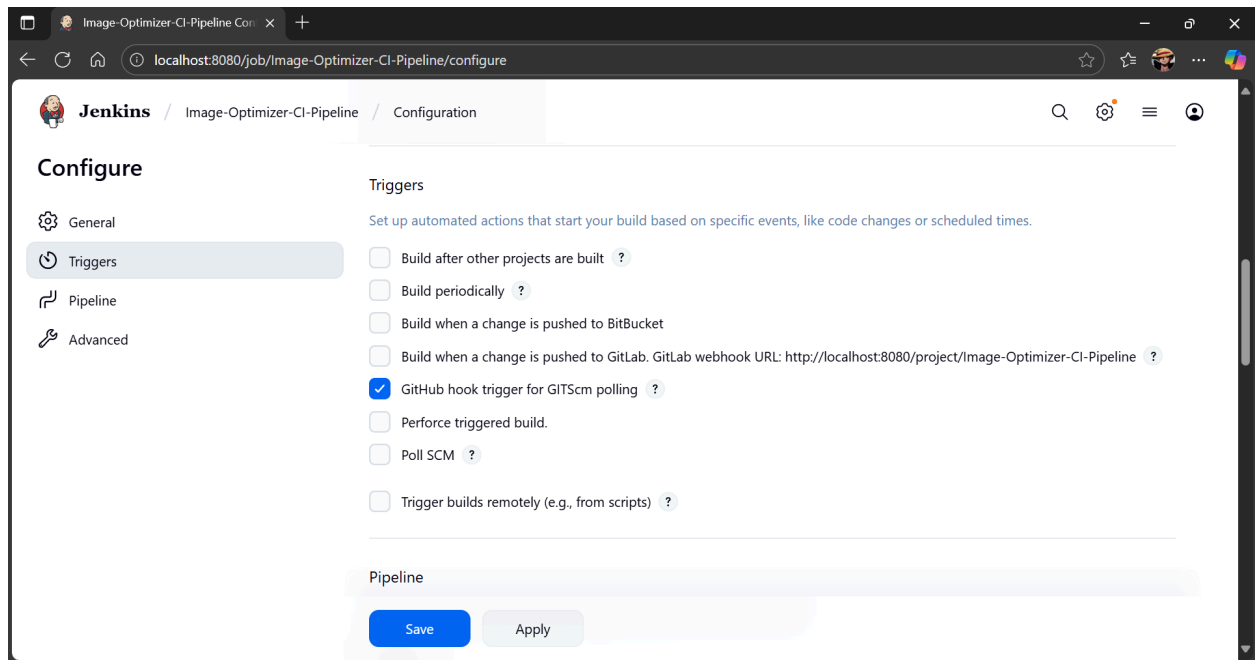


The screenshot shows the Jenkins 'New Item' page. The browser address bar indicates the URL is `localhost:8080/view/all/newJob`. The Jenkins header shows 'Jenkins / All / New Item'. The page title is 'New Item'. Below the title, there is a form to create a new item:

Enter an item name

Select an item type

- Freestyle project**
Classic, general-purpose job type that checks out from up to one SCM, executes build steps serially, followed by post-build steps like archiving artifacts and sending email notifications.
- Pipeline** (highlighted)
Orchestrates long-running activities that can span multiple build agents. Suitable for building pipelines (formerly known as workflows) and/or organizing complex activities that do not easily fit in free-style job type.
- Multi-configuration project**
Suitable for projects that need a large number of different configurations, such as testing on multiple environments, platform-specific builds, etc.



The screenshot shows the Jenkins configuration page for a job named 'Image-Optimizer-CI-Pipeline'. The 'Triggers' tab is selected in the left sidebar. The main area is titled 'Configure' and 'Triggers'. It contains a list of triggers with checkboxes. The 'GitHub hook trigger for GITScm polling' is checked. Below the triggers is a 'Pipeline' section with a 'Save' button and an 'Apply' button.

Configure

General
Triggers
Pipeline
Advanced

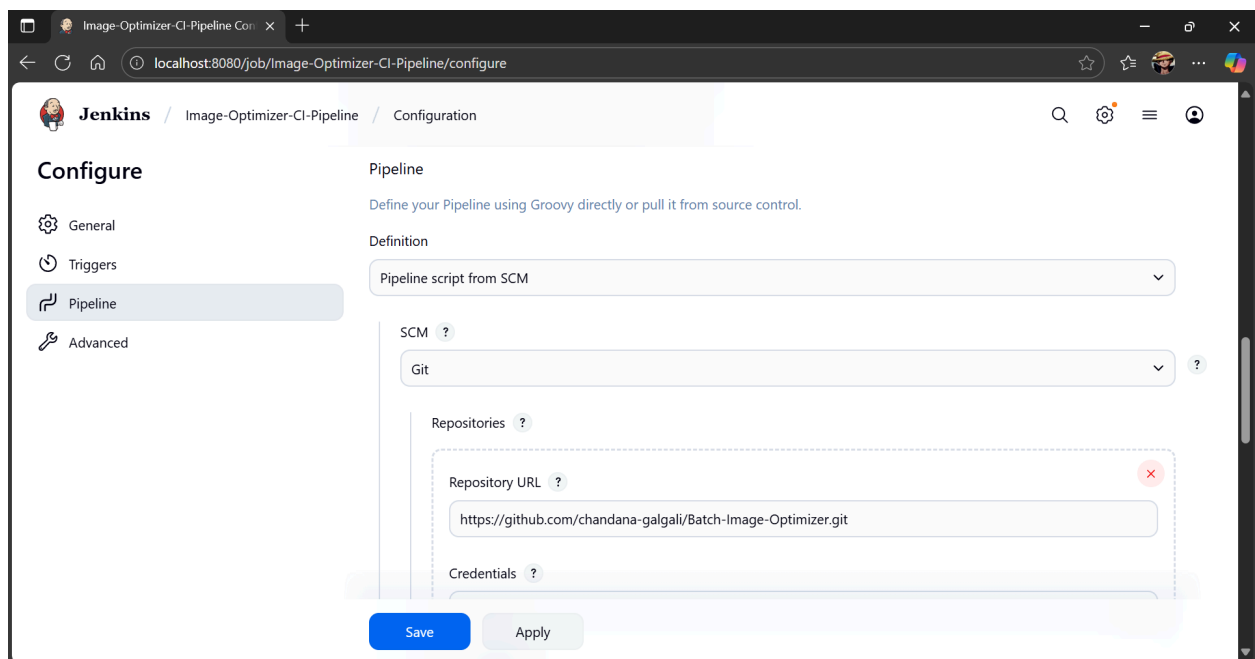
Triggers

Set up automated actions that start your build based on specific events, like code changes or scheduled times.

- ☐ Build after other projects are built ?
- ☐ Build periodically ?
- ☐ Build when a change is pushed to BitBucket
- ☐ Build when a change is pushed to GitLab. GitLab webhook URL: `http://localhost:8080/project/Image-Optimizer-CI-Pipeline` ?
- ☒ GitHub hook trigger for GITScm polling ?
- ☐ Perforce triggered build.
- ☐ Poll SCM ?
- ☐ Trigger builds remotely (e.g., from scripts) ?

Pipeline

Save Apply



The screenshot shows the Jenkins configuration page for a job named 'Image-Optimizer-CI-Pipeline'. The 'Pipeline' tab is selected in the left sidebar. The main area is titled 'Configure' and 'Pipeline'. It contains a 'Definition' section with a dropdown menu set to 'Pipeline script from SCM'. Below this is an 'SCM' section with a dropdown menu set to 'Git'. Under 'SCM', there is a 'Repositories' section with a 'Repository URL' field containing 'https://github.com/chandana-galgali/Batch-Image-Optimizer.git'. There is also a 'Credentials' section. At the bottom are 'Save' and 'Apply' buttons.

Configure

General
Triggers
Pipeline
Advanced

Pipeline

Define your Pipeline using Groovy directly or pull it from source control.

Definition

Pipeline script from SCM

SCM

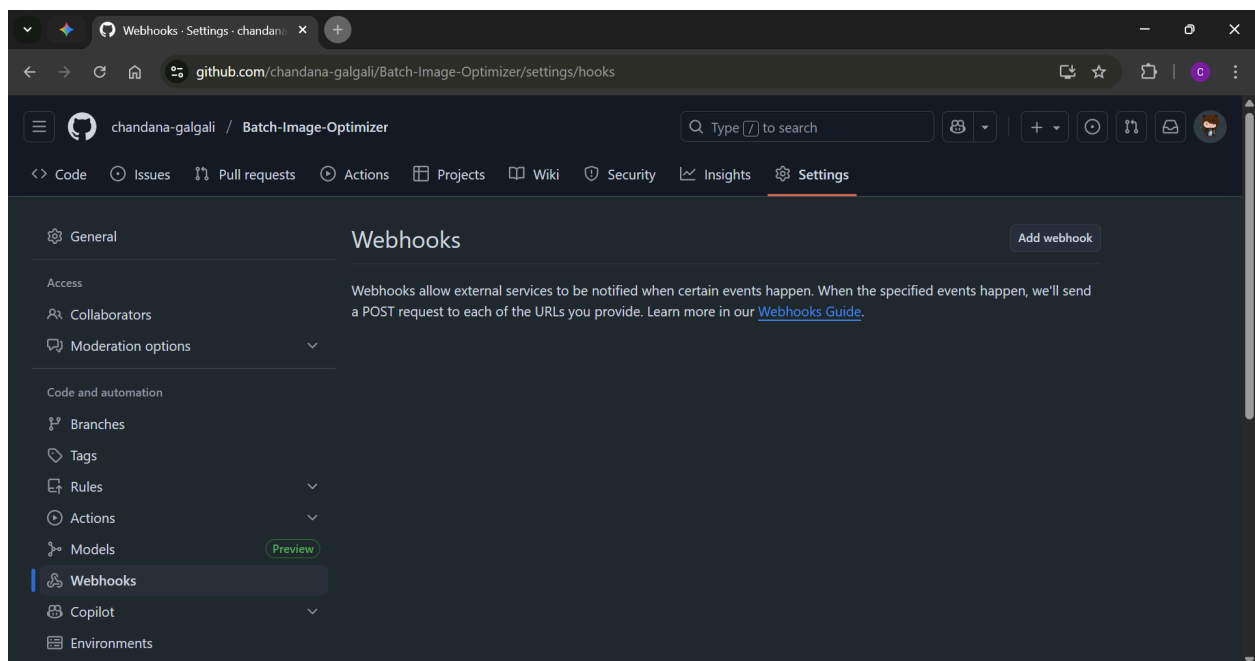
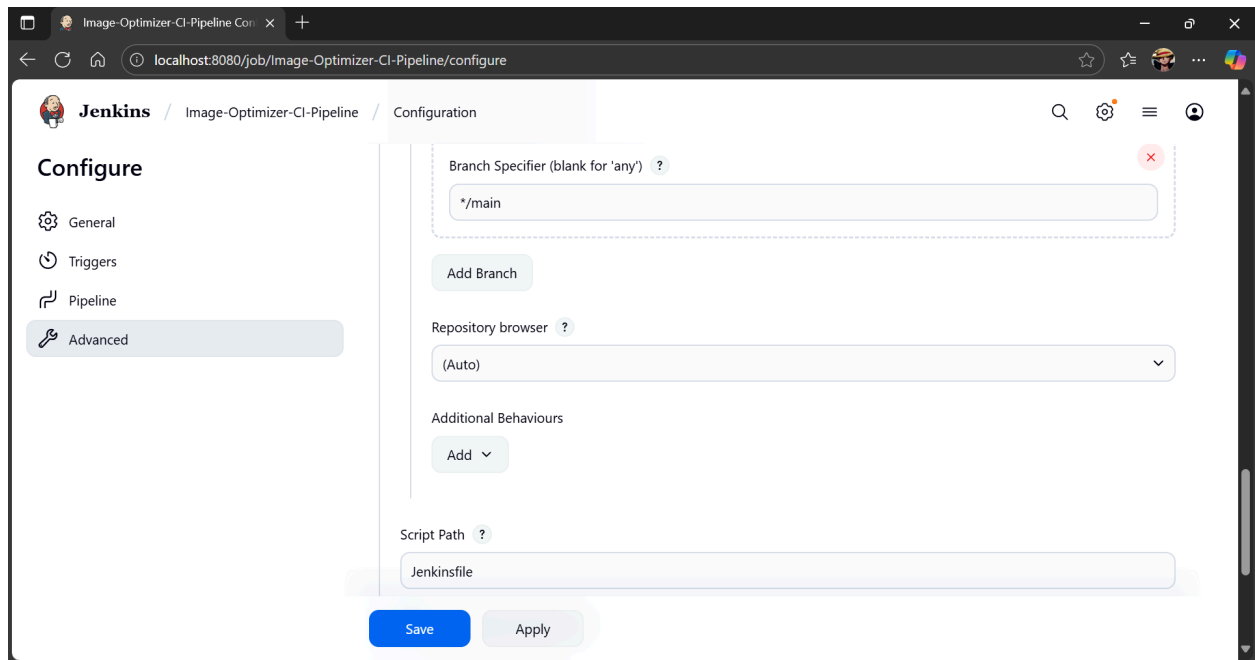
Git

Repositories

Repository URL ?
`https://github.com/chandana-galgali/Batch-Image-Optimizer.git`

Credentials

Save Apply



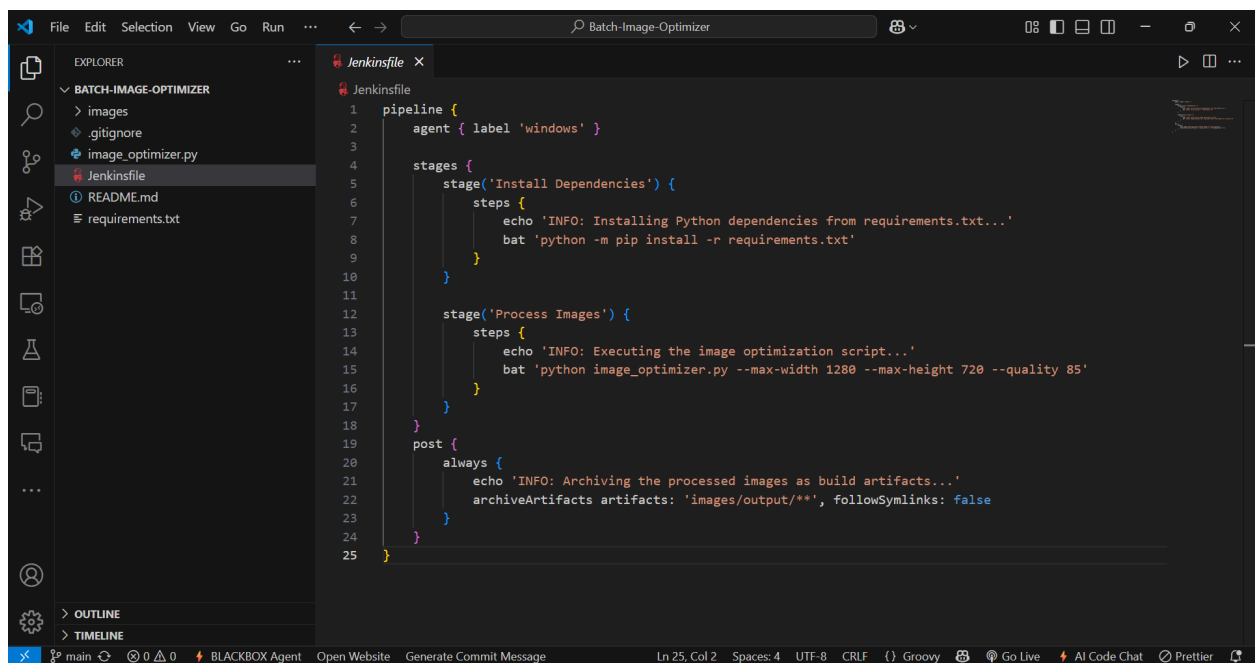
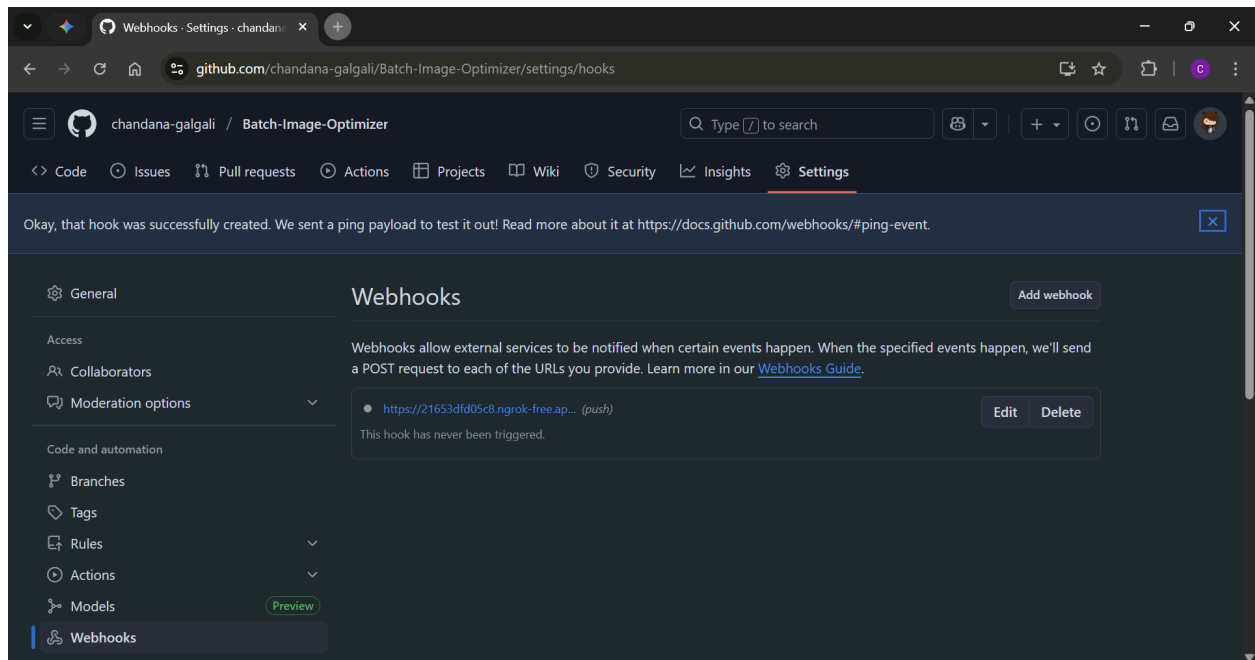
The screenshot shows the 'Add webhook' page in a web browser. The address bar displays 'github.com/chandana-galgali/Batch-Image-Optimizer/settings/hooks/new'. The page header includes the repository name and a search bar. The left sidebar contains navigation links: Code, Issues, Pull requests, Actions, Projects, Wiki, Security, Insights, and Settings (which is highlighted). The main content area is titled 'Webhooks / Add webhook' and contains the following fields:

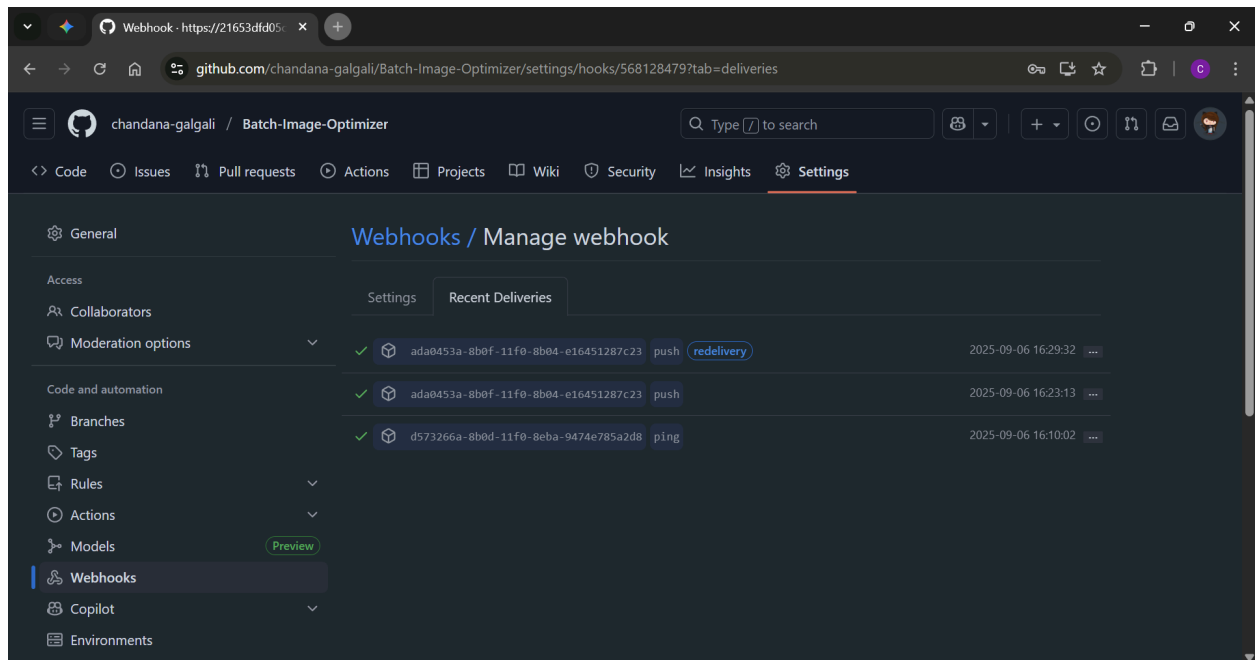
- General**: A section with links for Access, Collaborators, and Moderation options.
- Code and automation**: A section with links for Branches, Tags, Rules, Actions, Models (with a 'Preview' button), Webhooks (highlighted), Copilot, and Environments.
- Webhooks / Add webhook**: The main configuration area with the following fields:
 - Payload URL ***: A text input field containing 'https://21653dfd05c8.ngrok-free.app/github-webhook/'.
 - Content type ***: A dropdown menu set to 'application/json'.
 - Secret**: A text input field for a secret key.
 - SSL verification**: A section with a note 'By default, we verify SSL certificates when delivering payloads.'

This screenshot shows the lower portion of the 'Add webhook' page. The left sidebar is partially visible, showing 'Webhooks' highlighted under 'Code and automation', and other sections like 'Security' and 'Integrations'. The main content area includes:

- SSL verification**: A section with a note 'By default, we verify SSL certificates when delivering payloads.' and two radio buttons: 'Enable SSL verification' (selected) and 'Disable (not recommended)'.
- Which events would you like to trigger this webhook?**: A section with three radio buttons: 'Just the push event.' (selected), 'Send me everything.', and 'Let me select individual events.'.
- Active**: A checkbox labeled 'Active' which is checked, with a sub-note 'We will deliver event details when this hook is triggered.'
- Add webhook**: A green button to save the configuration.

The footer of the page contains the GitHub logo, copyright information '© 2025 GitHub, Inc.', and links for Terms, Privacy, Security, Status, GitHub Community, Docs, Contact, Manage cookies, and Do not share my personal information.





Webhook - https://21653dfd05c x +

github.com/chandana-galgali/Batch-Image-Optimizer/settings/hooks/568128479?tab=deliveries

chandana-galgali / Batch-Image-Optimizer

Type / to search

<> Code Issues Pull requests Actions Projects Wiki Security Insights Settings

General Webhooks / Manage webhook

Access

Collaborators

Moderation options

Code and automation

Branches

Tags

Rules

Actions

Models Preview

Webhooks

Copilot

Environments

Settings Recent Deliveries

Commit SHA	Event	Time	Actions
ada0453a-8b0f-11f0-8b04-e16451287c23	push	2025-09-06 16:29:32	redelivery
ada0453a-8b0f-11f0-8b04-e16451287c23	push	2025-09-06 16:23:13	
d573266a-8b0d-11f0-8eba-9474e785a2d8	ping	2025-09-06 16:10:02	

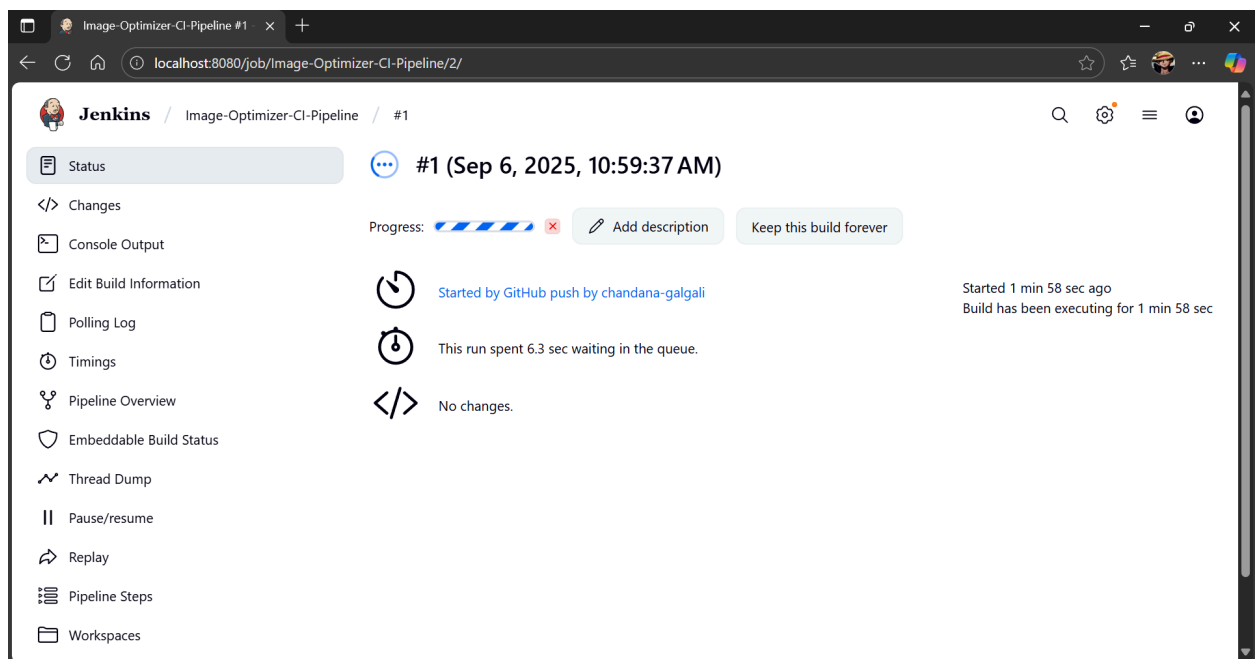


Image-Optimizer-CL-Pipeline #1 x +

localhost:8080/job/Image-Optimizer-CL-Pipeline/2/

Jenkins / Image-Optimizer-CL-Pipeline / #1

Status #1 (Sep 6, 2025, 10:59:37 AM)

Progress: Add description Keep this build forever

Started by GitHub push by chandana-galgali

Started 1 min 58 sec ago

Build has been executing for 1 min 58 sec

This run spent 6.3 sec waiting in the queue.

No changes.

Changes

Console Output

Edit Build Information

Polling Log

Timings

Pipeline Overview

Embeddable Build Status

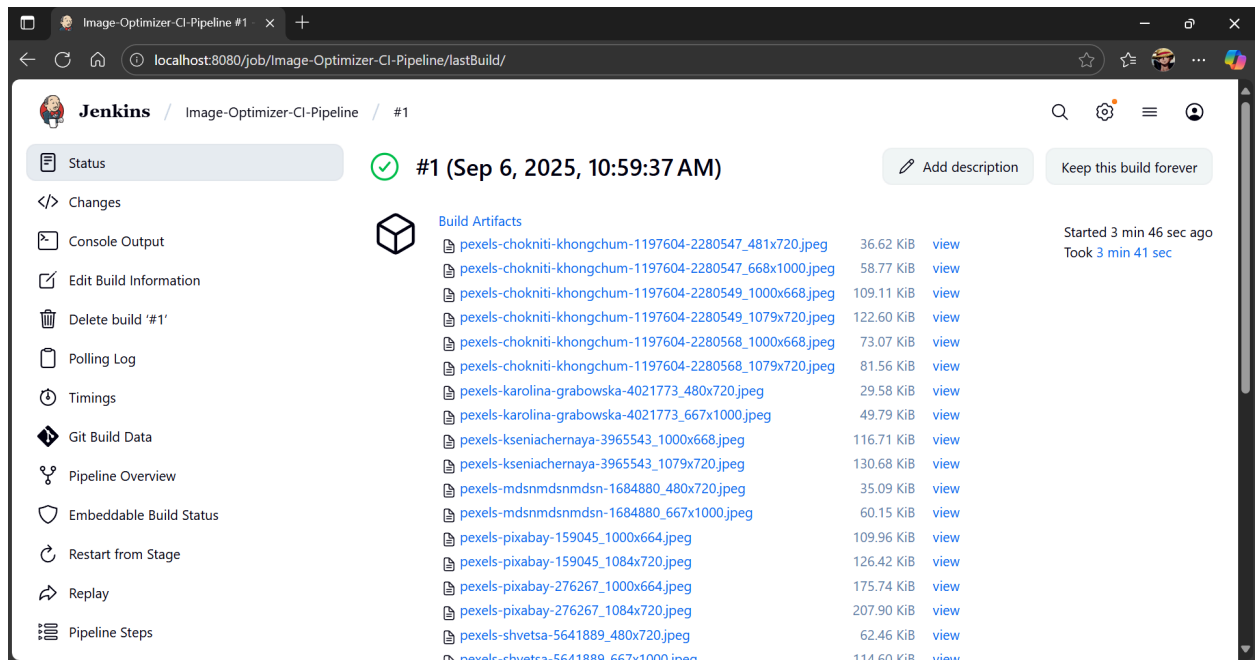
Thread Dump

Pause/resume

Replay

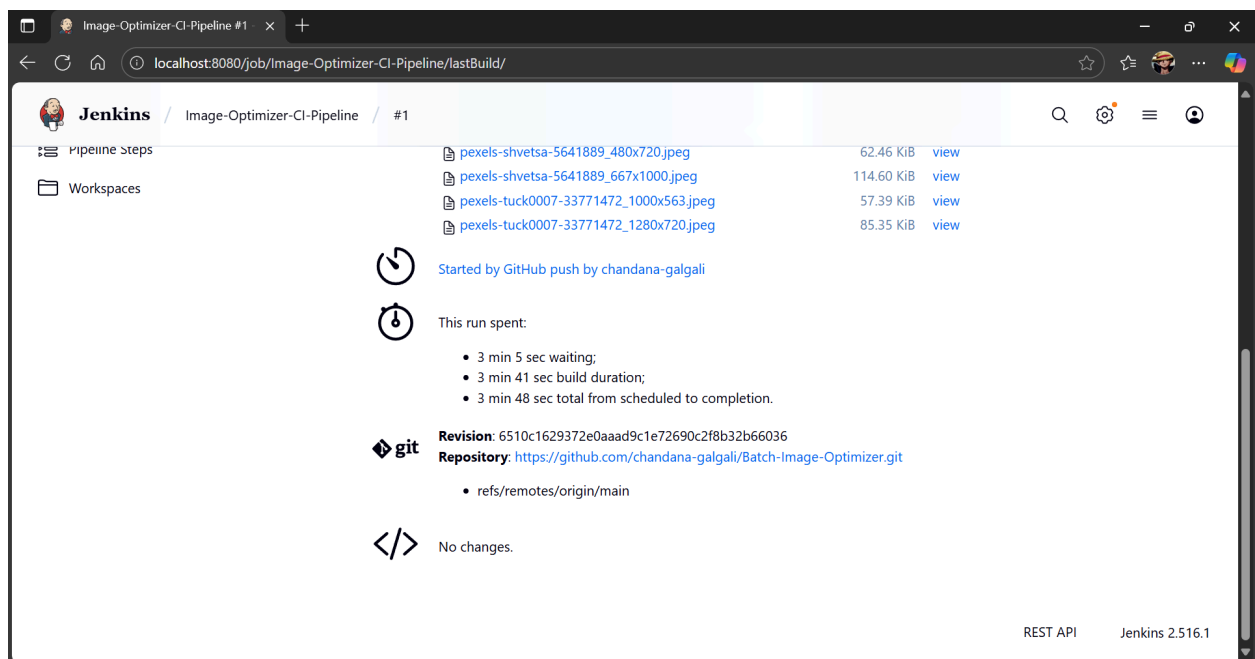
Pipeline Steps

Workspaces



This screenshot shows the Jenkins web interface for a build named '#1' of the 'Image-Optimizer-CI-Pipeline'. The build is successful, indicated by a green checkmark. The left sidebar contains navigation links for Status, Changes, Console Output, Edit Build Information, Delete build '#1', Polling Log, Timings, Git Build Data, Pipeline Overview, Embeddable Build Status, Restart from Stage, Replay, and Pipeline Steps. The main content area displays a list of build artifacts under the heading 'Build Artifacts'. Each artifact is a JPEG image with a unique filename, size in KiB, and a 'view' link. The artifacts include images from 'pexels-chokniti-khongchum', 'pexels-karolina-grabowska', 'pexels-kseniachernaya', 'pexels-mdsnmndsmndsn', 'pexels-pixabay', and 'pexels-shvetsa'. On the right, it shows the build started 3 minutes and 46 seconds ago and took 3 minutes and 41 seconds to complete. There are buttons for 'Add description' and 'Keep this build forever'.

Artifact Name	Size (KiB)	Action
pexels-chokniti-khongchum-1197604-2280547_481x720.jpeg	36.62	view
pexels-chokniti-khongchum-1197604-2280547_668x1000.jpeg	58.77	view
pexels-chokniti-khongchum-1197604-2280549_1000x668.jpeg	109.11	view
pexels-chokniti-khongchum-1197604-2280549_1079x720.jpeg	122.60	view
pexels-chokniti-khongchum-1197604-2280568_1000x668.jpeg	73.07	view
pexels-chokniti-khongchum-1197604-2280568_1079x720.jpeg	81.56	view
pexels-karolina-grabowska-4021773_480x720.jpeg	29.58	view
pexels-karolina-grabowska-4021773_667x1000.jpeg	49.79	view
pexels-kseniachernaya-3965543_1000x668.jpeg	116.71	view
pexels-kseniachernaya-3965543_1079x720.jpeg	130.68	view
pexels-mdsnmndsmndsn-1684880_480x720.jpeg	35.09	view
pexels-mdsnmndsmndsn-1684880_667x1000.jpeg	60.15	view
pexels-pixabay-159045_1000x664.jpeg	109.96	view
pexels-pixabay-159045_1084x720.jpeg	126.42	view
pexels-pixabay-276267_1000x664.jpeg	175.74	view
pexels-pixabay-276267_1084x720.jpeg	207.90	view
pexels-shvetsa-5641889_480x720.jpeg	62.46	view
pexels-shvetsa-5641889_667x1000.jpeg	114.60	view



This screenshot shows the Jenkins web interface for the same build '#1'. The left sidebar shows 'Pipeline Steps' and 'Workspaces'. The main content area displays the build's history, showing it was started by a GitHub push from 'chandana-galgali'. Below this, a summary of the run's duration is provided: 3 minutes 5 seconds waiting, 3 minutes 41 seconds build duration, and a total of 3 minutes 48 seconds from scheduled to completion. The build's revision and repository are also shown. At the bottom, it indicates 'No changes' were detected. The Jenkins version 2.516.1 and REST API endpoint are visible in the footer.

Started by GitHub push by chandana-galgali

This run spent:

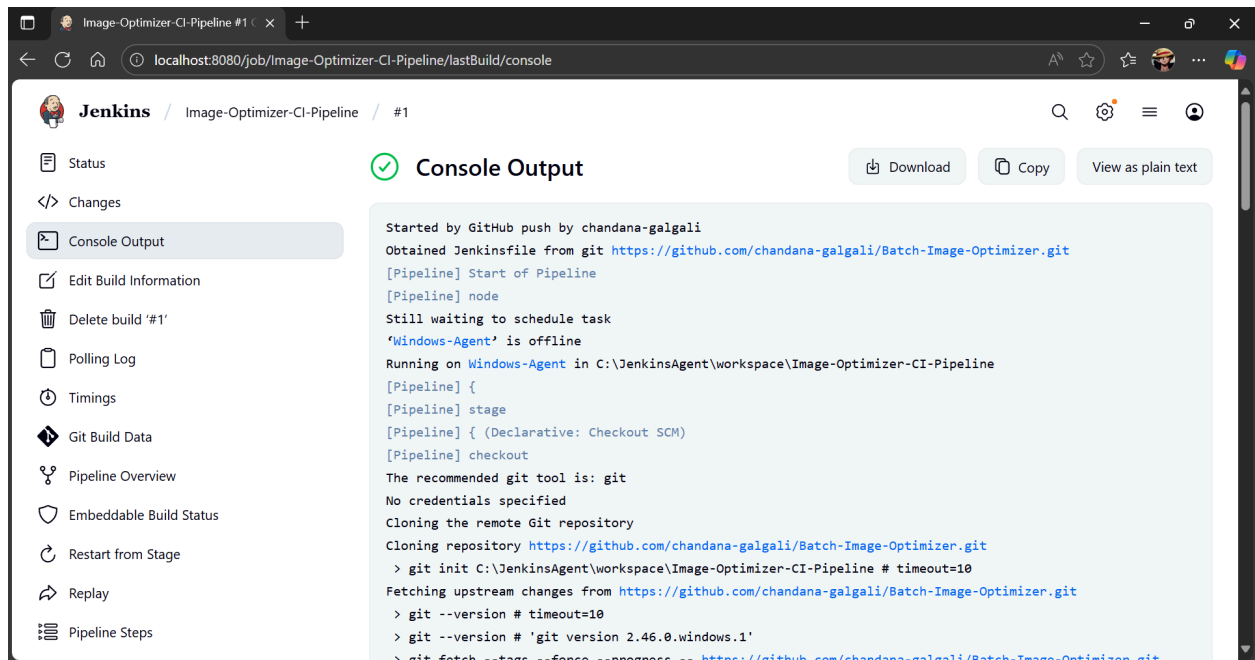
- 3 min 5 sec waiting;
- 3 min 41 sec build duration;
- 3 min 48 sec total from scheduled to completion.

Revision: 6510c1629372e0aad9c1e72690c2f8b32b66036
Repository: <https://github.com/chandana-galgali/Batch-Image-Optimizer.git>

- refs/remotes/origin/main

No changes.

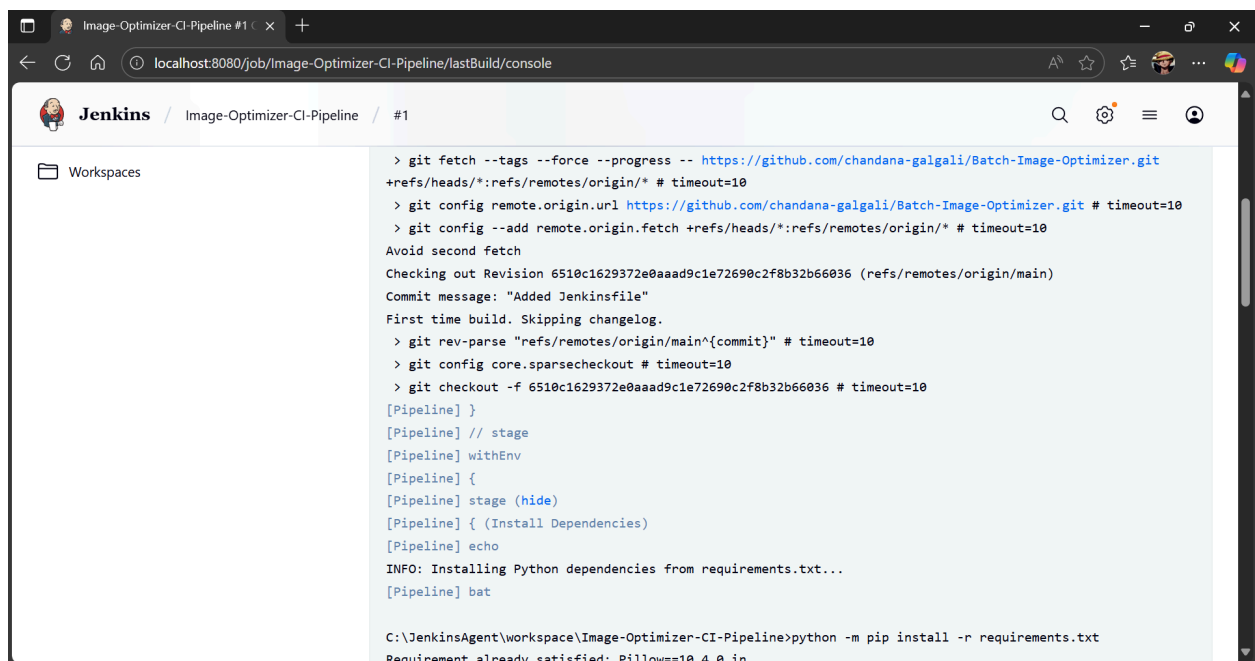
REST API Jenkins 2.516.1



The screenshot shows the Jenkins web interface for the 'Image-Optimizer-CI-Pipeline' job, build #1. The 'Console Output' tab is selected, displaying the execution log. The log indicates the pipeline was triggered by a GitHub push. It shows the start of the pipeline, a node check, and the execution of the 'Checkout' stage. The 'Checkout' stage is currently running on the 'Windows-Agent'.

Console Output

```
Started by GitHub push by chandana-galgali
Obtained Jenkinsfile from git https://github.com/chandana-galgali/Batch-Image-Optimizer.git
[Pipeline] Start of Pipeline
[Pipeline] node
Still waiting to schedule task
'Windows-Agent' is offline
Running on Windows-Agent in C:\JenkinsAgent\workspace\Image-Optimizer-CI-Pipeline
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Declarative: Checkout SCM)
[Pipeline] checkout
The recommended git tool is: git
No credentials specified
Cloning the remote Git repository
Cloning repository https://github.com/chandana-galgali/Batch-Image-Optimizer.git
> git init C:\JenkinsAgent\workspace\Image-Optimizer-CI-Pipeline # timeout=10
Fetching upstream changes from https://github.com/chandana-galgali/Batch-Image-Optimizer.git
> git --version # timeout=10
> git --version # 'git version 2.46.0.windows.1'
> git fetch --tags --force --progress -- https://github.com/chandana-galgali/Batch-Image-Optimizer.git
```

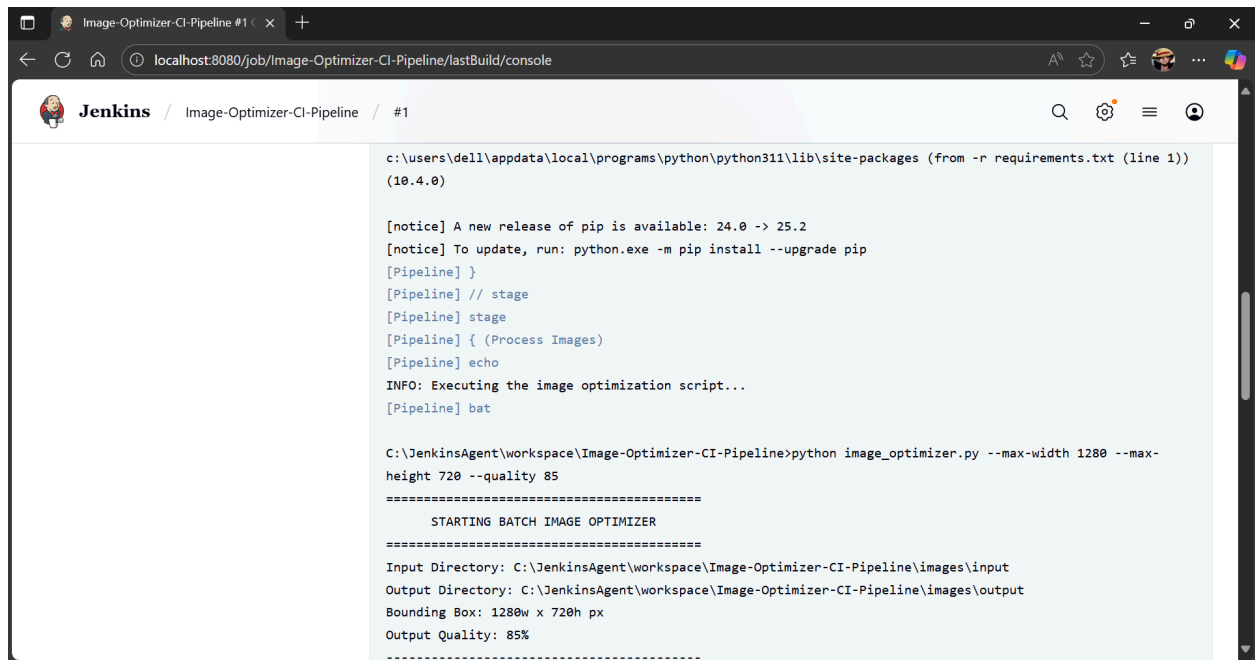


This screenshot continues the Jenkins console output from the previous image. It shows the continuation of the 'Checkout' stage, including the execution of 'git fetch' and 'git checkout' commands. The log shows the checkout of revision 6510c1629372e0aaad9c1e72690c2f8b32b66036. The 'First time build. Skipping changelog.' message is displayed. The 'git rev-parse' command is used to get the commit hash, and the 'git checkout' command is used to checkout the specific revision. The 'withEnv' block is shown, and the 'stage' is marked as 'hide'. The 'Install Dependencies' stage is shown, and the 'echo' command is used to display the message 'INFO: Installing Python dependencies from requirements.txt...'. The 'bat' command is used to execute the 'pip install' command.

Console Output

```
> git fetch --tags --force --progress -- https://github.com/chandana-galgali/Batch-Image-Optimizer.git
+refs/heads/*:refs/remotes/origin/* # timeout=10
> git config remote.origin.url https://github.com/chandana-galgali/Batch-Image-Optimizer.git # timeout=10
> git config --add remote.origin.fetch +refs/heads/*:refs/remotes/origin/* # timeout=10
Avoid second fetch
Checking out Revision 6510c1629372e0aaad9c1e72690c2f8b32b66036 (refs/remotes/origin/main)
Commit message: "Added Jenkinsfile"
First time build. Skipping changelog.
> git rev-parse "refs/remotes/origin/main^{commit}" # timeout=10
> git config core.sparsecheckout # timeout=10
> git checkout -f 6510c1629372e0aaad9c1e72690c2f8b32b66036 # timeout=10
[Pipeline] }
[Pipeline] // stage
[Pipeline] withEnv
[Pipeline] {
[Pipeline] stage (hide)
[Pipeline] { (Install Dependencies)
[Pipeline] echo
INFO: Installing Python dependencies from requirements.txt...
[Pipeline] bat

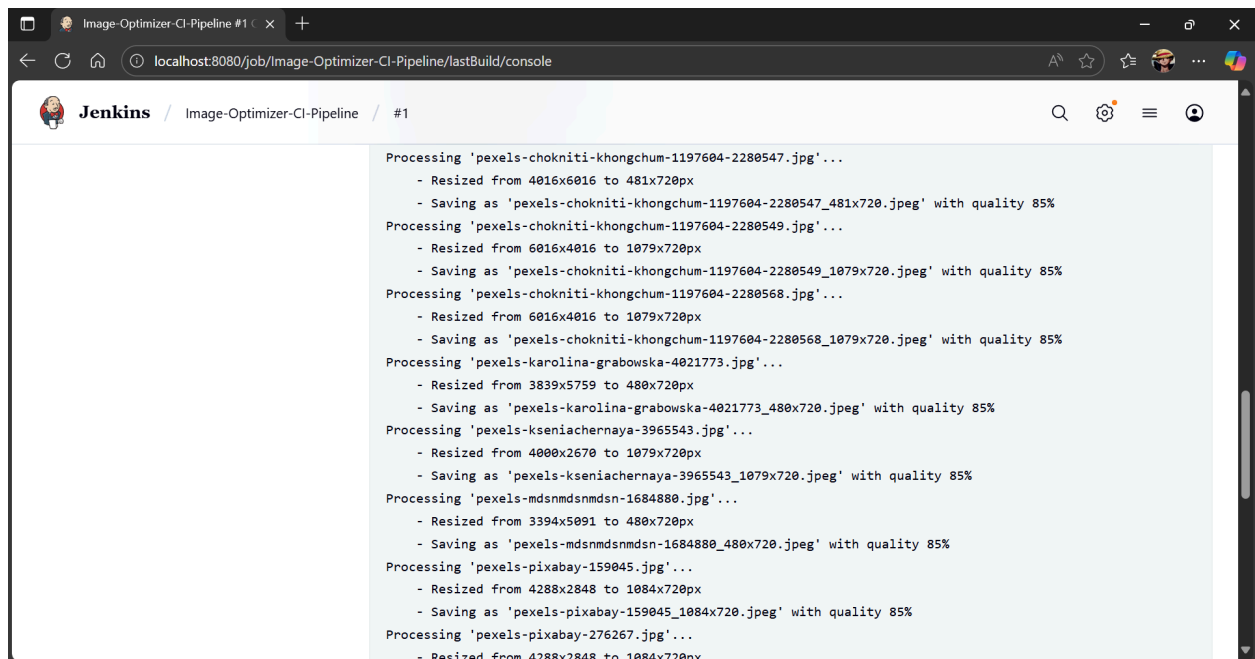
C:\JenkinsAgent\workspace\Image-Optimizer-CI-Pipeline>python -m pip install -r requirements.txt
Requirement already satisfied: Pillow==10.4.0 in
```



```
c:\users\dell\appdata\local\programs\python\python311\lib\site-packages (from -r requirements.txt (line 1))
(10.4.0)

[notice] A new release of pip is available: 24.0 -> 25.2
[notice] To update, run: python.exe -m pip install --upgrade pip
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Process Images)
[Pipeline] echo
INFO: Executing the image optimization script...
[Pipeline] bat

C:\JenkinsAgent\workspace\Image-Optimizer-CI-Pipeline>python image_optimizer.py --max-width 1280 --max-
height 720 --quality 85
=====
STARTING BATCH IMAGE OPTIMIZER
=====
Input Directory: C:\JenkinsAgent\workspace\Image-Optimizer-CI-Pipeline\images\input
Output Directory: C:\JenkinsAgent\workspace\Image-Optimizer-CI-Pipeline\images\output
Bounding Box: 1280w x 720h px
Output Quality: 85%
=====
```



```
Processing 'pexels-chokniti-khongchum-1197604-2280547.jpg'...
- Resized from 4016x6016 to 481x720px
- Saving as 'pexels-chokniti-khongchum-1197604-2280547_481x720.jpeg' with quality 85%
Processing 'pexels-chokniti-khongchum-1197604-2280549.jpg'...
- Resized from 6016x4016 to 1079x720px
- Saving as 'pexels-chokniti-khongchum-1197604-2280549_1079x720.jpeg' with quality 85%
Processing 'pexels-chokniti-khongchum-1197604-2280568.jpg'...
- Resized from 6016x4016 to 1079x720px
- Saving as 'pexels-chokniti-khongchum-1197604-2280568_1079x720.jpeg' with quality 85%
Processing 'pexels-karolina-grabowska-4021773.jpg'...
- Resized from 3839x5759 to 480x720px
- Saving as 'pexels-karolina-grabowska-4021773_480x720.jpeg' with quality 85%
Processing 'pexels-kseniachernaya-3965543.jpg'...
- Resized from 4000x2670 to 1079x720px
- Saving as 'pexels-kseniachernaya-3965543_1079x720.jpeg' with quality 85%
Processing 'pexels-mdsnmdsnmdsn-1684880.jpg'...
- Resized from 3394x5091 to 480x720px
- Saving as 'pexels-mdsnmdsnmdsn-1684880_480x720.jpeg' with quality 85%
Processing 'pexels-pixabay-159045.jpg'...
- Resized from 4288x2848 to 1084x720px
- Saving as 'pexels-pixabay-159045_1084x720.jpeg' with quality 85%
Processing 'pexels-pixabay-276267.jpg'...
- Resized from 4288x2848 to 1084x720px
```

```

- Saving as 'pexels-pixabay-276267_1084x720.jpeg' with quality 85%
Processing 'pexels-shvetsa-5641889.jpg'...
- Resized from 4160x6240 to 480x720px
- Saving as 'pexels-shvetsa-5641889_480x720.jpeg' with quality 85%
Processing 'pexels-tuck0007-33771472.jpg'...
- Resized from 4096x2304 to 1280x720px
- Saving as 'pexels-tuck0007-33771472_1280x720.jpeg' with quality 85%
-----
PROCESS COMPLETE
Successfully processed: 10 images
=====
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Declarative: Post Actions)
[Pipeline] echo
INFO: Archiving the processed images as build artifacts...
[Pipeline] archiveArtifacts (hide)
Archiving artifacts
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // withEnv

```

```

=====
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Declarative: Post Actions)
[Pipeline] echo
INFO: Archiving the processed images as build artifacts...
[Pipeline] archiveArtifacts
Archiving artifacts
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS

```

REST API Jenkins 2.516.1

Questions:

1. What is Jenkinsfile?

Ans: A Jenkinsfile is a text file that contains the definition of a Jenkins pipeline, which specifies the steps required to build, test, and deploy software in a continuous integration/continuous deployment (CI/CD) environment. The Jenkinsfile outlines the entire process, from fetching the code from the version control system (like GitHub) to performing build tasks, running tests, and deploying the software. This file is usually

stored within the project's repository and is executed automatically by Jenkins whenever there are changes or commits in the code. It helps automate the entire CI/CD process, reducing manual intervention and ensuring consistency across builds.

Outcomes: CO3 — Comprehend the effective code building and testing process

Conclusion: (Conclusion to be based on the Results and outcomes achieved)

In this experiment, we successfully integrated Jenkins with GitHub for Continuous Integration (CI) to automate the process of building, testing, and deploying software after each code commit. By configuring Jenkins with GitHub and using a Jenkinsfile to define the build process, we ensured that any changes pushed to the repository were automatically built and tested. This integration allowed us to catch potential errors early in the development cycle, reducing the time taken to identify and fix bugs. The implementation of CI practices enhanced the software development process by ensuring that only the best code was merged into the main branch, which led to more efficient development cycles and improved software quality.

Grade: AA / AB / BB / BC / CC / CD / DD

Signature of faculty in-charge with date

References:

Books/ Journals/ Websites:

1. <https://guides.github.com/introduction/git-handbook/>
 2. Jenkins Beginner Tutorial 8 - Jenkins integration with GIT (SCM):
<https://www.youtube.com/watch?v=bGqS0f4Utn4&t=7s>
 3. Integrate with GitHub: build after each commit (Get started with Jenkins, part 13):
<https://www.youtube.com/watch?v=Z3S2gMBUkBo>
 4. <https://www.guru99.com/continuous-integration.html>
-